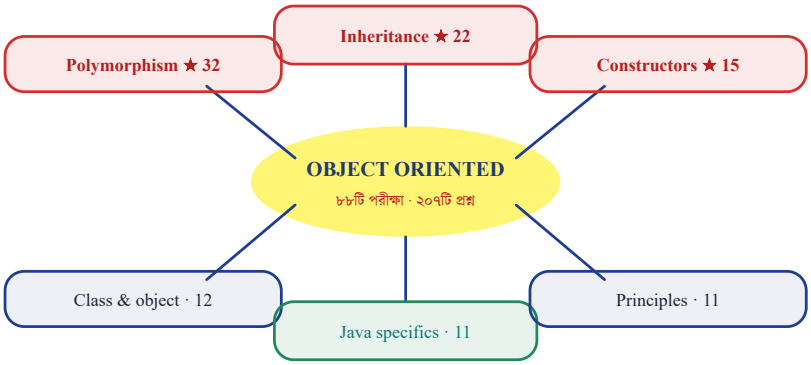
অবজেক্ট ওরিয়েন্টেড প্রোগ্রামিং

BCS · NTRCA · BPSC · ব্যাংক · বিদ্যুৎ খাত · BUET-প্রণীত প্রশ্নপত্রে আসা প্রতিটি OOP প্রশ্ন, ৩২ পাতায় সমাধান



↑ সংখ্যাটি হলো কয়টি ভিন্ন পরীক্ষায় বিষয়টি এসেছে (লাল = সবচেয়ে বেশি)

প্রশ্ন-কার্ড যেভাবে পড়বেন

> **প্র:** প্রশ্নপত্রে যেভাবে ছাপা (বানান ঠিক করা)।
> ✎ **যে পরীক্ষায় এসেছে:** যে পরীক্ষাগুলো থেকে নেওয়া, যেমন BPSC AP 2023 · 49th BCS ICT (2025)

এই পাতার প্রতিটি প্রোগ্রাম কম্পাইল করে চালানো হয়েছে; সবুজ = আসল আউটপুট। একই কোডে C++ ও Java কখনো মেশানো হয়নি।

সবচেয়ে বেশি এসেছে

① overloading বনাম overriding · ৮
② friend function · ৩
③ OOP ও এর স্তম্ভ · ২ + ২
④ class বনাম object · ২
⑤ উদাহরণসহ inheritance · ২
⑥ structured বনাম OOP · ২

সিলেবাসের যা আছে

"**অবজেক্ট ওরিয়েন্টেড ধারণা**: class, object, **encapsulation**, **inheritance**, **polymorphism**, constructor ও destructor, operator overloading, virtual function, template, exception handling; C++ ও Java", সঙ্গে thread ও design pattern।

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ করুন · **বেগুনি বাক্স** = মনে রাখার কৌশল · **লাল বাক্স** = পরীক্ষার ফাঁদ · **সবুজ** = যাচাই করা উত্তর

সূচিপত্র

✓ OOP, স্তম্ভ · structured বনাম OOP … পৃ. ২–৪
✓ Class ও object · class প্রোগ্রাম …… পৃ. ৫–৭
✓ Encapsulation · ব্যাংক প্রোগ্রাম …… পৃ. ৮–৯
✓ Access specifier · friend …… পৃ. ১০–১১
✓ Inheritance ১, ২ · প্রোগ্রাম …… পৃ. ১২–১৬
✓ Polymorphism · overloading …… পৃ. ১৭–১৯
✓ Virtual · interface · ফাঁদ …… পৃ. ২০–২২
✓ Constructor ১–৩ …… পৃ. ২৩–২৫
✓ Java, C++ · আউটপুট …… পৃ. ২৬–২৭
✓ ছোট প্রোগ্রাম …… পৃ. ২৮
✓ Thread · exception …… পৃ. ২৯–৩০
✓ Facade pattern · আগের রাত … পৃ. ৩১–৩২

১ OOP, কেন দরকার, চারটি স্তম্ভ

> **প্র১:** Object-Oriented Programming (OOP) বলতে কী বোঝায়? OOP কেন দরকার এবং এর মৌলিক ধারণাগুলো (Pillars) লিখুন। / অবজেক্ট ওরিয়েন্টেড প্রোগ্রামিং (OOP) বলতে কী বোঝায়? এর প্রাথমিক নীতিগুলো লিখুন।
> ✎ **যে পরীক্ষায় এসেছে:** 15th NTRCA · 17th NTRCA · 15th NTRCA ICT · BOF SAE(Computer) 2021 · 37th BCS

OOP: এমন একটি প্রোগ্রামিং পদ্ধতি যেখানে প্রোগ্রাম গড়া হয় **object** দিয়ে: প্রতিটি object তার **ডেটা** (attribute) ও সেই ডেটায় কাজ করা **method** একসঙ্গে রাখে, আর object-গুলো একে অপরের method ডেকে কাজ করে (**message passing**)। **Class** হলো যে নকশা থেকে object তৈরি হয়।

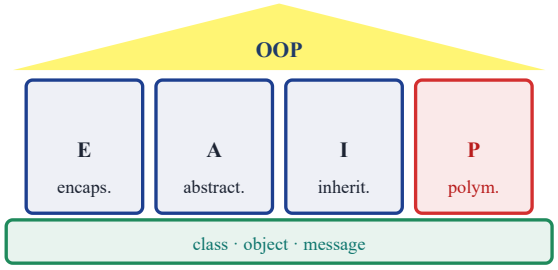
কেন দরকার: বড় procedural প্রোগ্রামে যেকোনো function শেয়ার করা ডেটা বদলাতে পারে, তাই bug ছড়িয়ে পড়ে আর কোড পুনর্ব্যবহার হয় শুধু কপি করে। OOP ডেটাকে object-এর ভেতরে **সুরক্ষিত** রাখে, inheritance দিয়ে কোড **পুনর্ব্যবহার** করে, পরিবর্তন **একটি class-এর** ভেতরে রাখে, আর বাস্তব জিনিসের (account, student) মতো গঠন দেয়; ফলে নকশা ও রক্ষণাবেক্ষণ সহজ।

স্তম্ভ	এক লাইনে লিখুন	উদাহরণ
Encapsulation	ডেটা + method **এক ইউনিটে**	`private balance` + `deposit()`
Abstraction	**কী** দেখাও, **কীভাবে** লুকাও	`car.start()`, ইঞ্জিন লুকানো
Inheritance	একটি class পুনর্ব্যবহার ও সম্প্রসারণ	`Savings extends Account`
Polymorphism	এক interface, **বহু রূপ**	প্রতি shape-এ `shape.area()`

প্র২: কী কী ধারণার উপর ভিত্তি করে OOP প্রতিষ্ঠিত? ধারণাগুলো ব্যাখ্যা করুন। / Object Oriented Programming-এর তিনটি প্রধান মূলনীতি ব্যাখ্যা করুন। / OOP-এর প্রধান বৈশিষ্ট্যগুলো কী?
✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA CSE (2023) · 17th NTRCA ICT (2023) · 16th NTRCA ICT · 37th BCS

চারটি স্তম্ভ লিখুন, তারপর **class**, **object**, **message passing**, **dynamic binding**। **তিনটি মূলনীতি** চাইলে লিখুন **encapsulation, inheritance, polymorphism** (abstraction তখন encapsulation-এর ভেতরে ধরা হয়)।

যেভাবে MCQ এসেছে	উত্তর	যে পরীক্ষায়
Object oriented programming কিসের উপর ভিত্তি করে?	**object** (ও class)	Combined Bank Officer(IT) 2026
Object oriented Programming-এর বৈশিষ্ট্য কোনটি? Polymorphism / Friend Function / Structure / Loop	**Polymorphism**	DoICT AP 2020



মনে রাখার কৌশল: "**A PIE**": **A**bstraction, **P**olymorphism, **I**nheritance, **E**ncapsulation। পাই খান, চারটি লিখুন।

পরীক্ষার ফাঁদ: Friend function হলো C++-এর একটি **সুবিধা**, OOP-এর **মূলনীতি** নয়; বরং এটি encapsulation দুর্বল করে।

২ একটি উদাহরণে OOP-এর সব ধারণা

প্র৩: উপযুক্ত উদাহরণসহ OOP ধারণাগুলো ব্যাখ্যা করুন (নিচে Java)। / Object Oriented Programming-এর যেকোনো দুটি বৈশিষ্ট্য উদাহরণসহ ব্যাখ্যা করুন। / ইনহেরিট্যান্স, পলিমরফিজম এবং এনক্যাপসুলেশন ব্যাখ্যা করুন। / Encapsulation, inheritance ও polymorphism কী?
✎ **যে পরীক্ষায় এসেছে:** JBL ANE 2022 · BPSC AP 2021 · 16th NTRCA CSE (2019) · 36th BCS (2016)

```
abstract class Shape {                  // abstraction: what, not how
    private String name;                // encapsulation: hidden data
    Shape(String n) { name = n; }
    String getName() { return name; }
    abstract double area();
```

```
}
class Circle extends Shape {            // inheritance: Circle IS-A Shape
    private double r;
    Circle(double r) { super("Circle"); this.r = r; }
    double area() { return Math.PI * r * r; }    // polymorphism
}
class Square extends Shape {
    private double s;
    Square(double s) { super("Square"); this.s = s; }
    double area() { return s * s; }
}
Shape[] all = { new Circle(1), new Square(2) };
for (Shape x : all) System.out.printf("%s %.2f%n", x.getName(), x.area());
```

→ Circle 3.14 · Square 4.00 (একই কল `x.area()`, দুই রকম আচরণ)

Encapsulation: `name`, `r`, `s` private। **Abstraction**: `Shape` বলে `area()` থাকবে, কীভাবে তা বলে না। **Inheritance**: দুটি shape-ই `getName()` পুনর্ব্যবহার করে। **Polymorphism**: লুপ প্রতিটি object-এর সঠিক `area()` চালায়।

> **প্র৪:** C++ একটি অবজেক্ট ওরিয়েন্টেড প্রোগ্রামিং—ব্যাখ্যা করুন।
> ✎ **যে পরীক্ষায় এসেছে:** 15th NTRCA Computer Operation (2019)

C++ প্রতিটি স্তম্ভ সমর্থন করে: `private/protected/public`-সহ `class` (encapsulation), pure virtual function-সহ abstract class (abstraction), পাঁচ রকম inheritance, আর overloading (compile time) ও `virtual` function (run time) দিয়ে polymorphism। তবে এটি **hybrid** ভাষা: class ছাড়া C-ধাঁচের কোডও কম্পাইল হয়, তাই Smalltalk-এর মতো "বিশুদ্ধ" OOP নয়।

> **প্র৫:** অবজেক্ট ওরিয়েন্টেড ডিজাইনের বৈশিষ্ট্যগুলো লিখ।
> ✎ **যে পরীক্ষায় এসেছে:** WZPDCL SAE 2023

Class বাস্তব জিনিসের মডেল · **information hiding** · inheritance · polymorphism · **message passing** · modularity · পুনর্ব্যবহার · **low coupling, high cohesion**।

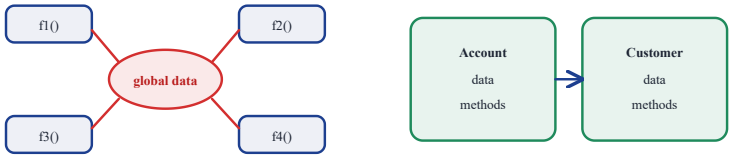
> **প্র৬:** OOP-এর মূলনীতিগুলো লিখুন। Polymorphism কী? ৩টি OOP ভাষার নাম লিখুন।
> ✎ **যে পরীক্ষায় এসেছে:** 40th BCS (2020) · DESCO SAE 2023

মূলনীতি: চারটি স্তম্ভ (পাতা ২)। Polymorphism: এক নাম, বহু রূপ (পাতা ১৭)। ভাষা: **C++, Java, Python** (এ ছাড়া C#, Smalltalk)।

৩ Structured (procedural) বনাম object-oriented

> **প্র৭:** Structured ও object-oriented programming-এর পার্থক্য লিখুন। / OOP ও Procedural Oriented Programming-এর পার্থক্য।
> ✎ **যে পরীক্ষায় এসেছে:** 45th BCS CSE (2024) · Combined Bank SO(AME) 2021 · Combined Bank AHE 2021

বিষয়	Structured / procedural (POP)	Object-oriented (OOP)
একক	**function**	**object** (class)
ডেটা ও কোড	আলাদা; ডেটা এদিক-ওদিক পাঠানো হয়	**একসঙ্গে বাঁধা**
ডেটার নিরাপত্তা	global ডেটা, লুকানো নেই	private ডেটা, access control
নকশা	top-down	bottom-up
পুনর্ব্যবহার	function কপি করে	**inheritance**, composition
overloading	নেই	function + operator overloading
যোগাযোগ	function call	message passing
পরিবর্তন	সবখানে ছড়াতে পারে	এক class-এই থাকে
উপযোগী	ছোট প্রোগ্রাম	বড়, বদলাতে থাকা সিস্টেম
উদাহরণ	**C, Pascal**, FORTRAN	**C++, Java**, C#, Python



বাঁয়ে: সব function শেয়ার করা ডেটায় · ডানে: প্রতিটি object নিজের ডেটা রাখে

প্র৮: Procedural Oriented ও Object Oriented Programming Languages-এর মধ্যে পার্থক্য কী? উভয় Language-এর ২টি করে উদাহরণ দিন।

✎ **যে পরীক্ষায় এসেছে:** BPSC AP 2021

টেবিলটি লিখুন, শেষে ঠিক দুটি করে: procedural **C, Pascal**; object-oriented **C++, Java**।

প্র৯: প্রচলিত structured programming-এর তুলনায় OOP-এর সুবিধাগুলো লিখুন।

✎ **যে পরীক্ষায় এসেছে:** Combined Bank Officer(IT) 2023 · 38th BCS (2018)

১. **পুনর্ব্যবহার**: কোড কপি না করে পরীক্ষিত class inherit করুন।
২. **ডেটার নিরাপত্তা**: private ডেটা বাইরে থেকে নষ্ট করা যায় না।
৩. **Modularity**: দল আলাদা আলাদা class বানায় ও পরীক্ষা করে।
৪. **সহজ রক্ষণাবেক্ষণ**: পরিবর্তন এক class-এর ভেতরে থাকে।
৫. **সম্প্রসারণ**: নতুন subclass যোগ, পুরোনো কোড চলে (polymorphism)।
৬. **বাস্তবের মডেল**: class সমস্যার সঙ্গে মেলে, নকশা ও debugging দ্রুত।

মনে রাখার কৌশল: Structured = "**আগে ক্রিয়া**"; OOP = "**আগে বিশেষ্য**"।

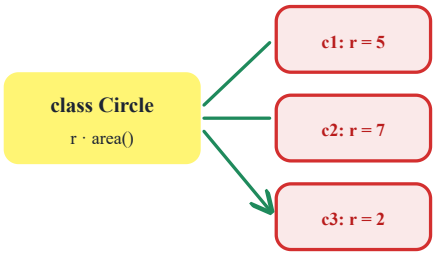
পরীক্ষার ফাঁদ: C++ পুরো OOP **নয়**, C মোটেই OOP **নয়**; C-এর `struct`-এ method নেই।

৪ Class ও object

প্র১০: উদাহরণসহ Class এবং Object-এর মধ্যে পার্থক্য ব্যাখ্যা করুন। / Class এবং Object কাকে বলে? কীভাবে Object তৈরি করতে হয়? / Object কী? কীভাবে Object তৈরি করতে হয়? / Class ও Object ডিক্লেয়ার করার নিয়ম লিখুন।

✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA CSE (2023) · 17th NTRCA ICT (2023) · 15th NTRCA ICT · 15th NTRCA · 17th NTRCA · 16th NTRCA ICT · 15th NTRCA CSE

Class	Object
একটি **নকশা** / user-defined type	class-এর একটি **instance**
যৌক্তিক; ডেটার জন্য মেমরি নেয় না	বাস্তব; মেমরি দখল করে
একবার ঘোষণা	যত খুশি তৈরি
keyword `class`	ঘোষণা বা `new` দিয়ে তৈরি
`class Circle`	`c1`, `c2`



এক class, অনেক object

```
class Circle {                      // declare: keyword class, name, { members };
    double r;                       // data member (private by default)
public:
    Circle(double radius) { r = radius; }
    double area() { return 3.1416 * r * r; }    // member function
};
int main() {
    Circle c1(5);                   // object on the stack
    Circle *c2 = new Circle(7);     // object on the heap (Java: always new)
    cout << c1.area() << " " << c2->area();    delete c2;
}
```

→ 78.54 153.938

প্র১১: C++-এ object কী? / Class-এর member function কীভাবে লেখা হয়? / Object Oriented Programming-এ method কী? Method কীভাবে ব্যবহৃত হয় আলোচনা করুন।

✎ **যে পরীক্ষায় এসেছে:** 37th BCS · 16th NTRCA ICT

Object হলো class type-এর একটি variable: এর আছে **state** (data member), **behaviour** (member function) ও **identity** (নিজস্ব ঠিকানা)। **Member function (method)** হলো class-এর ভেতরে ঘোষিত function; একে object-এর উপর ডাকা হয় (`c1.area()`) এবং সেই object-এর ডেটায় কাজ করে। Class-এর ভেতরে লিখলে তা নিজে থেকেই `inline`, বাইরে লিখলে **scope operator `::`** দিয়ে — `double Circle::area() { return 3.1416*r*r; }`।

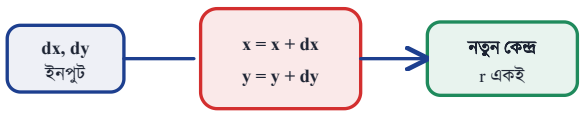
MCQ The class in C++ contains: (ক) data only (খ) function only (গ) data and function both (ঘ) None of the above → **(গ)** ✎ 49th BCS ICT (2025)

৫ Class-এর নকশা: ডেটা, method, object

প্র১২: একটি graphics package-এর সফটওয়্যারে circle class বানাতে হবে। Circle-কে তার মূলবিন্দু থেকে সরানো (translate) যাবে, আর এটি পরিধি ও ক্ষেত্রফল দেবে। Class-এর ডেটা ও method-এর প্রয়োজন চিহ্নিত করুন এবং translation method-এর data flow দেখান।

✎ **যে পরীক্ষায় এসেছে:** Combined Bank Officer(IT) 2024

ডেটা (private)	Method (public)
`x`, `y`: কেন্দ্র	`translate(dx, dy)`
`r`: ব্যাসার্ধ	`perimeter()` = $2\pi r$
	`area()` = πr^2
	constructor, `centre()`



পরিধি ও ক্ষেত্রফল বদলায় না

translate()-এর data flow

```
class Circle {                                  // Java
    private double x, y, r;
```

```
    Circle(double x, double y, double r) { this.x = x; this.y = y; this.r = r; }
    void translate(double dx, double dy) { x += dx; y += dy; }
    double perimeter() { return 2 * Math.PI * r; }
    double area() { return Math.PI * r * r; }
}
Circle c = new Circle(0, 0, 2);   c.translate(3, 4);
```

→ centre (3.0, 4.0) · perimeter 12.57 · area 12.57

প্র১৩: `Class test(){ Int Public: t(int value = 0){} t.getone(value) t.gettwo(value+10)} Int main(){ Test (5); cout<<"Get One"<<getone<<endl; cout<<"Get One"<<gettwo<<endl; }` (সঠিকভাবে লিখুন)

✎ **যে পরীক্ষায় এসেছে:** Dhaka WASA AME(Network) 2025

```
class Test {
    int value;
public:
    Test(int v = 0) { value = v; }
    int getOne() { return value; }
    int getTwo() { return value + 10; }
};
int main() {
    Test t(5);
    cout << "Get One " << t.getOne() << endl;
    cout << "Get Two " << t.getTwo() << endl;
}
```

পরীক্ষার ফাঁদ: ছাপা অবস্থায় এটি কম্পাইল হয় না: `Class`/`Int`/`Public` বড় হাতের, class-এর নাম চাই, `test()` নয়; constructor-এর নাম class-এর নামই হবে; member ডাকতে হয় object দিয়ে (`t.getOne()`)।

→ Get One 5 · Get Two 15

৬ Class প্রোগ্রাম ও C++-এর default

প্র১৪: C++-এ `student` নামে একটি class লিখুন যার তিনটি attribute name, roll, marks; get method ব্যবহার করে দুটি object তৈরি করুন এবং যার marks বেশি তার নাম প্রিন্ট করুন।

✎ **যে পরীক্ষায় এসেছে:** Cadet College Lecturer(ICT) 2025

```
class Student {
    string name; int roll; double marks;
public:
    Student(string n, int r, double m) { name = n; roll = r; marks = m; }
    string getName() { return name; }   double getMarks() { return marks; }
};
int main() {
    Student s1("Nusrat", 1, 78.5), s2("Tanvir", 2, 84);
    Student top = (s1.getMarks() > s2.getMarks()) ? s1 : s2;
    cout << "Highest: " << top.getName() << " (" << top.getMarks() << ")";
}
```

→ Highest: Tanvir (84)

প্র১৫: Structure ও class-এর পার্থক্য লিখুন।

✎ **যে পরীক্ষায় এসেছে:** BCIC AP 2025

C++-এ দুটিতেই ডেটা ও function রাখা যায়; পার্থক্য default-এ: `struct`-এর member ও inheritance স্বাভাবিকভাবে **public**, `class`-এর **private**। সাধারণ ডেটার জন্য `struct`, লুকানো ডেটাসহ object-এর জন্য `class`। (C-এর `struct`-এ function থাকে না; C#-এ `struct` value type।)

প্র১৬: Inner class-এর object তৈরি।
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. May 2019

```
class Outer {                                  // Java
    int x = 5;
    class Inner { int twice() { return x * 2; } }     // inner
    static class Nested { int one() { return 1; } }   // static nested
}
Outer o = new Outer();
Outer.Inner in = o.new Inner();       // needs an Outer object
Outer.Nested n = new Outer.Nested();  // does not
System.out.println(in.twice() + " " + n.one());
```

→ 10 1

(non-static) **inner class**-এর object একটি outer object-এর ভেতরে থাকে এবং তার field পড়তে পারে, তাই তৈরি হয় **outer.new Inner()** দিয়ে। **Static nested** class-এর জন্য outer object লাগে না।

প্র১৭: protected, private ও inheritance নিয়ে C++-এর ৫টি MCQ (প্রশ্নগুলো সংরক্ষিত নেই)।
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. April 2017

এগুলো **inheritance mode**-এর টেবিল যাচাই করে: base-এর member derived class-এ কী access পায়।

base member →	public	protected	private
`: public Base`	public	protected	access নেই
`: protected Base`	protected	protected	access নেই
`: private Base`	private	private	access নেই

৭ Encapsulation ও abstraction

প্র১৮: Abstraction ও encapsulation সংক্ষেপে বর্ণনা করুন।
✎ **যে পরীক্ষায় এসেছে:** BCC Programmer 2019

	Encapsulation	Abstraction
অর্থ	ডেটা + method **এক ইউনিটে** মুড়ে access সীমিত করা	শুধু **প্রয়োজনীয়** বৈশিষ্ট্য দেখানো, খুঁটিনাটি লুকানো
লুকায়	**ডেটা** (state)	**implementation** (জটিলতা)
স্তর	implementation	design
উপায়	`private` + getters/setters	abstract class, interface
উদাহরণ	`balance` বদলায় শুধু `deposit()` দিয়ে	ATM-এর "withdraw" বোতাম

প্র১৯: Abstraction ব্যবহার করে কীভাবে সমস্যার সমাধান করা যায়?
✎ **যে পরীক্ষায় এসেছে:** 37th BCS

① ব্যবহারকারীকে কী করতে হবে তা লিখুন, কীভাবে তা নয় · ② সেটি **interface / abstract class** হিসেবে লিখুন: `interface Payment { void pay(double amt); }` · ③ প্রতিটি উপায় আলাদা class-এ লুকান (`BKash`, `Card` `pay()` implement করে) · ④ প্রোগ্রাম শুধু `Payment` ব্যবহার করে, তাই Nagad যোগ করলে আর কিছু বদলায় না।

প্র২০: Encapsulation ও inheritance বিস্তারিত ব্যাখ্যা করুন। / OOP-তে encapsulation ও inheritance কীভাবে সুবিধাজনক ব্যাখ্যা করুন।
✎ **যে পরীক্ষায় এসেছে:** 38th BCS (2018) · Combined Bank Officer(IT) 2024

Encapsulation ডেটাকে তার ব্যবহারকারী method-এর সঙ্গে বাঁধে এবং `private` দিয়ে লুকায়; বাইরের কোড যাচাইকৃত public method ব্যবহার করে। লাভ: **ডেটার শুদ্ধতা**, ভেতরটা স্বাধীনভাবে বদলানো যায়, ব্যবহার সহজ। **Inheritance**-এ derived class base class-এর member পায় এবং নিজের member যোগ বা override করে (IS-A)। লাভ: **কোড পুনর্ব্যবহার**, কম পরীক্ষা, স্বাভাবিক hierarchy, run-time polymorphism।

যেভাবে MCQ এসেছে	উত্তর	যে পরীক্ষায়
ডেটা ও কোড একসঙ্গে বাঁধা?	**Encapsulation**	49th BCS ICT (2025)
Object-এর কিছু বিবরণে access সীমিত করতে ___ ব্যবহৃত হয়	**Encapsulation**	49th BCS CSE (2025)
Object-এর ভেতরের implementation লুকানোর কৌশল ("abstraction" অপশন নেই)	**Encapsulation**	Combined Bank ADA 2021
কোনটি encapsulation অর্জন করে না? private / class / getter-setter / global variable	**global variables**	BB AD(ICT) 2025

মনে রাখার কৌশল: **Abstraction** = **মেনু** (কী অর্ডার করা যায়)। **Encapsulation** = **রান্নাঘরের দরজা** (ভেতরে ঢোকা যায় না)।

পরীক্ষার ফাঁদ: "**Access** সীমিত" → encapsulation; "শুধু **প্রয়োজনীয়** বৈশিষ্ট্য দেখাও" → abstraction। ক্রিয়াটি পড়ুন।

৮ দুটি ব্যাংক-অ্যাকাউন্ট প্রোগ্রাম

প্র২১: যেকোনো object-oriented ভাষায় (C++/Java/Python) একটি ব্যাংক অ্যাকাউন্টের প্রোগ্রাম লিখুন: `BankAccount` class-এ থাকবে হিসাবধারীর নাম, হিসাব নম্বর ও balance; টাকা জমা, টাকা উত্তোলন (পর্যাপ্ত balance নিশ্চিত করে) ও হিসাবের বিবরণ দেখানোর member function। Data member private রেখে এবং সেগুলো পড়া ও বদলানোর public method দিয়ে encapsulation দেখান।

১০ নম্বর

✎ **যে পরীক্ষায় এসেছে:** Combined Bank Officer(IT) 2025

```
class BankAccount {                                        // Java
    private String holder, accountNo;  private double balance;   // hidden
    public BankAccount(String holder, String accountNo, double balance) {
        this.holder = holder; this.accountNo = accountNo; this.balance = balance;
    }
    public void deposit(double amt) { if (amt > 0) balance += amt; }
    public boolean withdraw(double amt) {
        if (amt > 0 && amt <= balance) { balance -= amt; return true; }
        System.out.println("Insufficient balance"); return false;
    }
    public double getBalance() { return balance; }
    public void display() { System.out.println(accountNo + " " + holder + " " + balance); }
}
// in main():
BankAccount a = new BankAccount("Karim", "0101-77", 5000);
a.deposit(1500);  a.withdraw(2000);  a.withdraw(9000);  a.display();
```

→ Insufficient balance · 0101-77 Karim 4500.0

নম্বর যেখানে: field **private** · `withdraw` যাচাই করে `amt <= balance` · `balance`-এর getter আছে, setter নেই · `a.balance = -500;` কম্পাইল হয় না।

প্র২২: Python-এ `BankAccount` নামে একটি class বানান। এতে name ও balance থাকবে এবং তিনটি কাজ: ১. জমা ২. উত্তোলন ৩. বর্তমান balance দেখানো।

✎ **যে পরীক্ষায় এসেছে:** BUET AP 2025

```
class BankAccount:
    def __init__(self, name, balance=0):
        self.name = name
```

→ Insufficient balance
→ Rahim: 1200

```
        self.__balance = balance      # private (name mangling)
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient balance")
    def show_balance(self):
        print(f"{self.name}: {self.__balance}")
acc = BankAccount("Rahim", 1000)
acc.deposit(500);  acc.withdraw(300)
acc.withdraw(5000); acc.show_balance()
```

`__init__` হলো Python-এর **constructor**; `self` হলো object নিজে (`this`-এর মতো)। সামনে দুটি underscore থাকায় **name mangling**-এ `__balance` private: `acc.__balance` লিখলে `AttributeError`।

মনে রাখার কৌশল: প্রতিটি ব্যাংক প্রশ্নের উত্তর একই **তিন-যাচাই**: amount > 0 · amount ≤ balance · ডেটা private।

৯ Access specifier: public, protected, private

প্র২৩: C++-এর বিভিন্ন access specifier কী কী? উদাহরণসহ এদের উদ্দেশ্য ব্যাখ্যা করুন।
✎ **যে পরীক্ষায় এসেছে:** BPSC ADA 2022

Access specifier ঠিক করে কে class-এর member ব্যবহার করতে পারবে। `public`: সবাই · `protected`: class নিজে, তার friend ও **derived class** · `private`: শুধু class ও তার friend (C++ `class`-এ default)।

```
class Account {
private:   double balance = 0;        // only Account (and its friends)
protected: double rate = 0.05;        // + classes derived from Account
public:    void deposit(double t) { if (t > 0) balance += t; }   // everyone
           double get() { return balance; }
};
class Savings : public Account {
public: double yearly() { return get() * rate; }   // rate: OK, balance: error
};
Savings s;  s.deposit(1000);  cout << s.get() << " " << s.yearly();
```

→ 1000 50

প্র২৪: Java-তে তিনটি access specifier আছে। কোনটি Public, Private ও Protected mode-এ কাজ করে তা বের করুন; হ্যাঁ হলে Y, না হলে N লিখুন। কলাম: class-এর ভেতরে · package-এর ভেতরে · package-এর বাইরে শুধু subclass · package-এর বাইরে।
✎ **যে পরীক্ষায় এসেছে:** Petrobangla AM(CSE) 2024

Specifier	Class-এর ভেতরে	Package-এর ভেতরে	Package-এর বাইরে, subclass	Package-এর বাইরে
`public`	**Y**	**Y**	**Y**	**Y**
`protected`	**Y**	**Y**	**Y**	**N**
`private`	**Y**	**N**	**N**	**N**
(default, keyword নেই)	Y	Y	N	N

পরীক্ষার ফাঁদ: একটি ছাপা সমাধান **identity matrix** (একই নামের "mode"-এ শুধু Y); এতে কিছুই বলা হয় না। উত্তর: **public YYYY · protected YYYN · private YNNN**.

যেভাবে MCQ এসেছে	উত্তর	যে পরীক্ষায়
C++-এ method-এ কোন modifier প্রয়োগ করা যায় না? Protected / Private / Public / Abstract	**Abstract**	BPSC AP 2022
C++-এ default inheritance কী? Public / Private / Protected / Compile time error	**Private**	BPSC AME 2019

`abstract` C++-এর keyword নয় (compiler বলে "unknown type name 'abstract'"); C++-এ method abstract হয় `= 0` দিয়ে। `class B : A {}` হলো **private** inheritance, `struct B : A {}` public।

১০ Friend function

প্র২৫: Friend Function কী? Friend Function-এর বৈশিষ্ট্য এবং সুবিধা ও অসুবিধাগুলো লিখুন।
✎ **যে পরীক্ষায় এসেছে:** 16th NTRCA · 16th NTRCA ICT · 18th NTRCA College ICT (2024)

Friend function: এমন একটি non-member function যাকে কোনো class তার ভেতরে `friend` keyword দিয়ে ঘোষণা করে; তখন সেটি ওই class-এর **private ও protected** member ব্যবহার করতে পারে।

বৈশিষ্ট্য

- ✓ class-এর ভেতরে ঘোষিত, `friend` বা `::` ছাড়া **বাইরে সংজ্ঞায়িত**
- ✓ **member নয়**: `this` নেই, class-এর scope-এ নেই
- ✓ সাধারণ function-এর মতো ডাকা হয়: `show(b)`, `b.show()` নয়
- ✓ object পায় **argument** হিসেবে, ব্যবহার করে `obj.member`
- ✓ friendship **inherit হয় না**, পারস্পরিক নয়, transitive নয়

সুবিধা

- ✓ একটি function **দুটি class-এর** private ডেটায় কাজ করতে পারে
- ✓ object-এর জন্য `operator<<` / `>>`-এ দরকার
- ✓ অনেক getter লাগে না

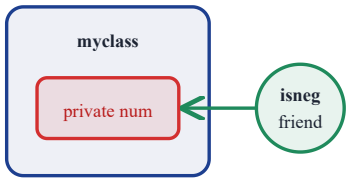
অসুবিধা

- ✓ **encapsulation** ভাঙে
- ✓ class-এর নিজের যাচাই এড়িয়ে যায়
- ✓ coupling বাড়ে, রক্ষণাবেক্ষণ কঠিন

প্র২৬: Friend function কী? নিচের class-এ `isneg()` নামে একটি friend function যোগ করে দেখান, যা `myclass` ধরনের একটি parameter নেয় এবং `num` ঋণাত্মক হলে true, না হলে false দেয়। `class myclass { int num; public: myclass(int i) { num = i; } };`
✎ **যে পরীক্ষায় এসেছে:** Hydrocarbon Unit AD(ICT) 2025

```
class myclass {
    int num;
public:
    myclass(int i) { num = i; }
    friend bool isneg(myclass ob);   // declared inside
};
bool isneg(myclass ob) {             // defined outside,
    return ob.num < 0;               // reads private num
}
int main() {
    myclass a(-5), b(8);
    cout << boolalpha << isneg(a) << " " << isneg(b);
}
```

→ true false



class-এর বাইরে, তবু দেয়ালের ভেতরে

Friend class: A-র ভেতরে `friend class B;` লিখলে B-এর **প্রতিটি** member function A-র private member ব্যবহার করতে পারে; friend function দরজা খোলে **একটি** function-এর জন্য।

মনে রাখার কৌশল: বন্ধুর কাছে **বাড়ির চাবি** আছে, কিন্তু সে **পরিবারের** নয়: `this` নেই, inheritance নেই।

পরীক্ষার ফাঁদ: `a.isneg()` বা `bool myclass::isneg` লিখলে নম্বর কাটা যায়; friend member নয়।

১১ Inheritance: সংজ্ঞা, প্রকার, সুবিধা-অসুবিধা

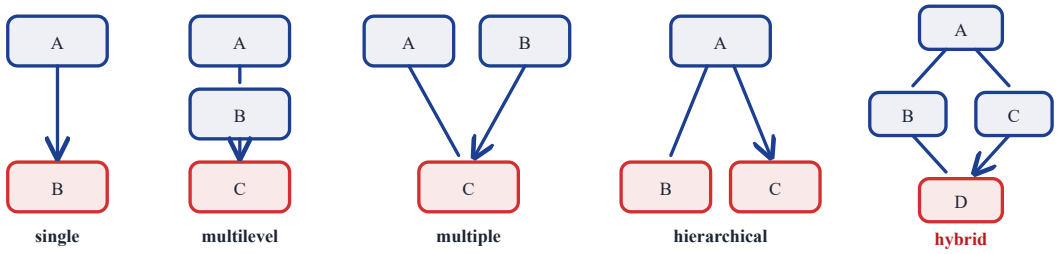
প্র২৭: Inheritance কী? উদাহরণসহ ব্যাখ্যা করুন। / Inheritance-এর বিভিন্ন প্রকারভেদ, সুবিধা ও অসুবিধা আলোচনা করুন। / Inheritance-এর ধারণা ব্যাখ্যা করুন; প্রচলিত তিনটি প্রকারের নাম ও সংক্ষিপ্ত বর্ণনা দিন।

✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA CSE (2023) · 17th NTRCA ICT (2023) · 17th NTRCA · Combined Bank AP 2026 · Bank (name cut off) AP

প্র২৮: যেকোনো object-oriented ভাষায় inheritance একটি গুরুত্বপূর্ণ বিষয়; এর প্রধান সুবিধা কোড পুনর্ব্যবহার। Inheritance-এর বিভিন্ন প্রকার সংক্ষেপে ব্যাখ্যা করুন।

✎ **যে পরীক্ষায় এসেছে:** Combined Bank Officer(IT) 2020

Inheritance: বিদ্যমান class (**base / parent**) থেকে নতুন class (**derived / child**) তৈরি, যাতে derived class base-এর member পায় এবং নিজের member যোগ বা override করে। এটি **IS-A** সম্পর্ক: Savings account *একটি* Account।



পাঁচটি প্রকার (তীর = "যে inherit করে"); প্রচলিত তিনটি: single, multilevel, hierarchical

```
class A {};
class B : public A {};              // single:       A → B
class C : public A {};              // hierarchical: A → B and A → C
class D : public C {};              // multilevel:   A → C → D
class E : public B, public C {};    // multiple:     B + C → E   (Java: only via interfaces)
class F : public D, public E {};    // hybrid:       two kinds at once
```

সুবিধা

- ✓ **কোড পুনর্ব্যবহার**: পরীক্ষিত কোড আবার লিখতে হয় না
- ✓ কম কোড, কম পরীক্ষা
- ✓ স্বাভাবিক hierarchy, সহজে বাড়ানো যায়
- ✓ run-time polymorphism সম্ভব হয়

অসুবিধা

- ✓ **tight coupling**: base বদলালে child ভাঙে
- ✓ গভীর hierarchy বোঝা কঠিন
- ✓ multiple inheritance → **diamond problem**
- ✓ child অব্যবহৃত base member বয়ে বেড়ায়

প্র২৯: Hybrid inheritance-এ কয়টি class ব্যবহার করা যায়?

✎ **যে পরীক্ষায় এসেছে:** BCC AP 2023

ধরে নেওয়া হলো: কোনো **ঊর্ধ্বসীমা নেই**। সর্বনিম্ন সংখ্যা নির্ভর করে কোন দুই ধরন মেশানো হয়েছে তার উপর: A → B এবং (A, B) → C-তে মাত্র ৩টি লাগে, আর উপরের পাঠ্যবইয়ের diamond-এ (hierarchical + multiple) **৪টি**। চিত্রসহ দুটি সংখ্যাই লিখুন।

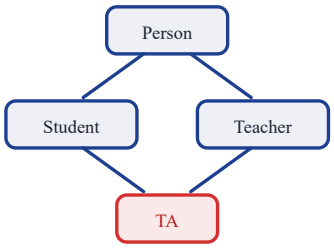
১২ Diamond, constructor-এর ক্রম, overriding

প্র৩০: C++-এ multiple inheritance-এর diamond problem কী?

✎ **যে পরীক্ষায় এসেছে:** BPSC AP 2023

`Student` ও `Teacher` দুটিই `Person` inherit করলে এবং `TA` দুটিকেই inherit করলে, একটি `TA` object-এ `Person`-এর **দুটি কপি** থাকে। তখন `t.id` **দ্ব্যর্থক**: compiler বলে "non-static member 'id' found in multiple base-class subobjects"। সমাধান: **virtual inheritance**, এতে একটিই শেয়ার করা `Person` থাকে।

```
class Person { public: int id = 7;
               Person() { cout << "Person "; } };
class Student : virtual public Person {
    public: Student() { cout << "Student "; } };
class Teacher : virtual public Person {
    public: Teacher() { cout << "Teacher "; } };
class TA : public Student, public Teacher {
    public: TA() { cout << "TA\n"; } };
int main() { TA t;  cout << t.id; }
```



→ Person Student Teacher TA
→ 7 (একটিই Person)

MCQ Class C, B-কে inherit করে, B আবার A-কে। C-এর object তৈরি করলে constructor কোন ক্রমে চলে? (a) C, B, A (b) A, C, B (c) A, B, C (d) C, A, B → **(c) প্রথমে A, তারপর B, শেষে C**; destructor চলে উল্টো ক্রমে (~C ~B ~A), চালিয়ে যাচাই করা। ✎ BUET IICT April 2021

প্র৩১: কোডসহ inheritance ও polymorphism-এর সংজ্ঞা লিখুন। / Java-তে inheritance ও polymorphism ব্যাখ্যা করুন; method overriding দেখাতে একটি Java প্রোগ্রাম লিখুন।
✎ **যে পরীক্ষায় এসেছে:** Combined Bank AP 2021 · Combined Bank Officer(IT) 2026

```
class Animal {
    void sound() { System.out.println("Animal makes a sound"); }
}
class Dog extends Animal {                  // inheritance
    @Override void sound() { System.out.println("Dog barks"); }
}
class Cat extends Animal {
    @Override void sound() { System.out.println("Cat meows"); }
}
// main():
Animal[] a = { new Dog(), new Cat(), new Animal() };
for (Animal x : a) x.sound();               // polymorphism
```

→ Dog barks
→ Cat meows
→ Animal makes a sound

Inheritance: `Dog` পায় `Animal`-এর সবকিছু। **Polymorphism**: `Animal` reference চালায় **object-এর** নিজের `sound()`, run time-এ ঠিক হয়। `@Override` দিলে compiler signature যাচাই করে।

মনে রাখার কৌশল: Constructor বাড়ি বানায় **ভিত থেকে উপরে** (A → B → C); destructor ভাঙে **ছাদ থেকে**।

পরীক্ষার ফাঁদ: Java-তে class-এর multiple inheritance **নেই**, তাই class-এর diamond নেই; একটি class অনেক interface implement করতে পারে।

১৩ Inheritance প্রোগ্রাম: super-এর কাজ

প্র৩২: Java প্রোগ্রামটি সম্পূর্ণ করুন: class A-তে আছে `alpha`, `beta` ও `display()`; class `Gamma` A-কে extend করে `gamma` যোগ করে। এর constructor লিখুন, `display()` override করুন এবং এমন `main` লিখুন যা `Gamma(3, 30, 10)` তৈরি করে।
✎ **যে পরীক্ষায় এসেছে:** BCC AP 2021

```
class A {
    int alpha, beta;
    A(int alpha, int beta) { this.alpha = alpha; this.beta = beta; }
    void display() { System.out.println("Alpha" + alpha);
                     System.out.println("Beta" + beta); }
}
class Gamma extends A {
```

→ Alpha3
→ Beta30
→ Gamma10

```
    int gamma;
    Gamma(int alpha, int beta, int gamma) {
        super(alpha, beta);             // base part first
        this.gamma = gamma;
    }
    @Override
    void display() { super.display(); System.out.println("Gamma" + gamma); }
}
public class Main {
    public static void main(String[] args) { new Gamma(3, 30, 10).display(); }
}
```

পরীক্ষার ফাঁদ: ছাপা সমাধানটি কম্পাইল হয় না: লেখা আছে `@override` (হবে **@Override**) এবং `public class main(` (হবে {)।

`super(...)` constructor-এর **প্রথম** statement হতেই হবে; `super.display()` parent-এর প্রিন্ট পুনর্ব্যবহার করে।

প্র৩৩: Java class `Rectangle` বিবেচনা করুন (private `length`, `width`, `height`; length ও width সেট করার একটি method; `getArea()` দেয় length*width)। `Rectangle`-কে inherit করে একই method ব্যবহার করে এমন class `Box` বানান; বাড়তি method বা variable ঘোষণা করবেন না।

✎ **যে পরীক্ষায় এসেছে:** BAERA Engineer(CSE) January 2021

```
class Rectangle {
    private double length, width;
    public void setValue(double length, double width) {
        this.length = length; this.width = width; }
    public double getArea() { return length * width; }
}
class Box extends Rectangle { }      // nothing new: all inherited
// main():
Box b = new Box();  b.setValue(4, 2.5);
System.out.println(b.getArea());
```

→ 10.0

ছাপা `getvalue double` হিসেবে ঘোষিত কিন্তু কিছু return করে না, তাই এটি `void setValue` হিসেবে লেখা। `Box` **private** field সরাসরি ছুঁতে পারে না; inherit করা **public method** দিয়ে পৌঁছায়।

১৪ Base + derived class প্রোগ্রাম

প্র৩৪: OOP নীতি মেনে একটি প্রোগ্রাম লিখুন। (a) ৪ base class `Employee`: private/protected Employee ID, Employee Name ও Basic Salary, একটি constructor এবং তথ্য দেখানোর method। (b) ৫ derived class `PermanentEmployee`: basic-এর উপর ২০% ভাতা, মোট বেতন হিসাব এবং তথ্যসহ মোট বেতন দেখানোর method। (c) ৪ `main()`-এ তিনটি `PermanentEmployee` object তৈরি করুন, প্রত্যেকের বিবরণ ও মোট বেতন দেখান এবং সর্বোচ্চ মোট বেতনের কর্মীকে দেখান।

✎ **যে পরীক্ষায় এসেছে:** ICB Securities AP 2026(Aptitude)

```
class Employee {
    protected int id;  protected String name;  protected double basic;
    Employee(int id, String name, double basic) { this.id = id; this.name = name; this.basic = basic; }
    void display() { System.out.println(id + " " + name + " basic " + basic); }
}
class PermanentEmployee extends Employee {
    PermanentEmployee(int id, String name, double basic) { super(id, name, basic); }
    double allowance() { return basic * 0.20; }
    double total() { return basic + allowance(); }
    void displayTotal() { display(); System.out.println("  total " + total()); }
}
public class Main {
    public static void main(String[] args) {
        PermanentEmployee[] e = { new PermanentEmployee(101, "Asif", 40000),
            new PermanentEmployee(102, "Mitu", 52000), new PermanentEmployee(103, "Rony", 45000) };
```

```
        PermanentEmployee top = e[0];
        for (PermanentEmployee p : e) { p.displayTotal(); if (p.total() > top.total()) top = p; }
        System.out.println("Highest: " + top.name + " " + top.total());
    }
}
```

→ 101 Asif basic 40000.0 / total 48000.0 · 102 Mitu … total 62400.0 · 103 Rony … total 54000.0 · **Highest: Mitu 62400.0**

প্র৩৫: `Bicycle` নামে একটি class লিখুন, যার constructor integer member `cadence`, `gear` ও `speed` initialise করে। আরেকটি class `MountainBike` লিখুন যা `Bicycle`-কে extend করে, নিজের constructor রাখে, integer field `seatHeight` যোগ করে এবং সেটি সেট করার method রাখে।

✎ **যে পরীক্ষায় এসেছে:** DESCO AE(CSE) 2016

```
class Bicycle {
    int cadence, gear, speed;
    Bicycle(int cadence, int gear, int speed) {
        this.cadence = cadence; this.gear = gear; this.speed = speed; }
}
class MountainBike extends Bicycle {
    int seatHeight;
    MountainBike(int cadence, int gear, int speed, int seatHeight) {
        super(cadence, gear, speed);
        this.seatHeight = seatHeight; }
    void setHeight(int h) { seatHeight = h; }
}
MountainBike m = new MountainBike(60, 3, 20, 28);  m.setHeight(30);
```

→ 60 3 20 30

মনে রাখার কৌশল: Child constructor = "**আগে বাবা, পরে আমি**": প্রথমে `super(...)`, পরে নিজের field।

১৫ Virtual function-সহ class hierarchy

প্র৩৬: এমন একটি C/C++ প্রোগ্রাম লিখুন যাতে class `Account` এবং subclass `SavingsAccount`, `CurrentAccount` ইত্যাদি সঠিক hierarchy-তে থাকে।

✎ **যে পরীক্ষায় এসেছে:** Combined Bank AP 2021

```
class Account {
protected:  string owner;  double balance;
public:
    Account(string o, double b) : owner(o), balance(b) {}
    void deposit(double amt) { if (amt > 0) balance += amt; }
    virtual bool withdraw(double amt) {
        if (amt > 0 && amt <= balance) { balance -= amt; return true; }  return false; }
    virtual void monthEnd() {}
    void show() { cout << owner << ": " << balance << "\n"; }
    virtual ~Account() {}
};
class Savings : public Account {                // earns interest
    double rate;
public:
    Savings(string o, double b, double r) : Account(o, b), rate(r) {}
    void monthEnd() override { balance += balance * rate / 12; }
};
class Current : public Account {                // may overdraw up to a limit
    double overdraft;
public:
    Current(string o, double b, double od) : Account(o, b), overdraft(od) {}
    bool withdraw(double amt) override {
```

```
        if (amt > 0 && amt <= balance + overdraft) { balance -= amt; return true; }  return false; }
};
int main() {
    Account* acc[2] = { new Savings("Rina", 12000, 0.06), new Current("Babul", 5000, 3000) };
    acc[0]->withdraw(2000);  acc[0]->monthEnd();          // 10000 + 50 interest
    acc[1]->withdraw(7000);                               // allowed: within the overdraft
    cout << boolalpha << acc[1]->withdraw(2000) << "\n";  // -4000 > limit: refused
    for (Account* a : acc) { a->show(); delete a; }
}
```

→ false · Rina: 10050 · Babul: -2000

পরীক্ষার ফাঁদ: `withdraw`-এ `virtual` না থাকলে `acc[1]->withdraw()` **base**-এর নিয়ম চালাত এবং overdraft দিত না। Base pointer দিয়ে `delete` করলে base class-এ **virtual destructor** দিন।

যেকোনো "base + derived" প্রোগ্রাম পাঁচ ধাপে

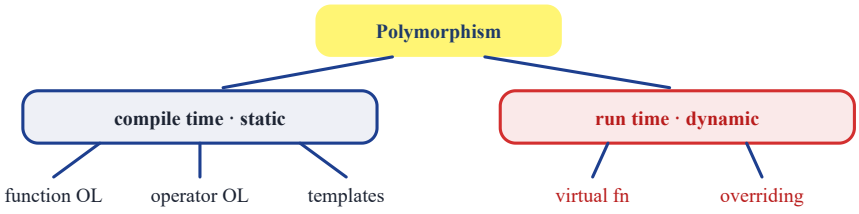
১. Base class: **সাধারণ** ডেটা (`protected`, বা `private` + getter) ও একটি constructor।
২. সাধারণ method base-এ; যে method আলাদা হবে তা `virtual` (C++) / override-যোগ্য (Java)।
৩. প্রতিটি derived class: তার **বাড়তি** ডেটা, আর এমন constructor যা আগে base-এরটি ডাকে (`: Base(...)` / `super(...)`)।
৪. শুধু যা আলাদা তা override করুন (`override` / `@Override`)।
৫. `main`: base-type array বা pointer-এ বিভিন্ন object, একটি loop-এ একই method ডাকা।

১৬ Polymorphism ও এর দুই প্রকার

প্র৩৭: Polymorphism কী? উদাহরণসহ এর বিভিন্ন প্রকার আলোচনা করুন। / Polymorphism বলতে কী বোঝেন? উদাহরণসহ বর্ণনা করুন। / উদাহরণসহ OOP-তে polymorphism-এর ধারণা ব্যাখ্যা করুন। / What is Polymorphism? Java language-এর আলোকে ব্যাখ্যা কর।

✎ **যে পরীক্ষায় এসেছে:** Combined Bank AP 2024 · Parliament Secretariat Programmer 2023 · MoFood Network/Website Manager 2025 · BTCL JAM 2022

Polymorphism: ("বহু রূপ") একই নাম — function, operator বা reference — **argument-এর type** বা **আসল object** অনুযায়ী ভিন্ন আচরণ করে: `+` সংখ্যা যোগ করে আবার string জোড়ে; প্রতি shape-এ `shape.area()`।



OL = overloading

	Compile-time	Run-time
কে ঠিক করে	compiler, argument-এর type দেখে	JVM / vtable, **object** দেখে
কীভাবে হয়	**overloading**, template	**overriding** + base reference
Java-তে	শুধু overloading (operator overloading নেই)	overriding; method স্বাভাবিকভাবেই virtual

প্র৩৮: Polymorphism কী? Compile time ও run time polymorphism দেখাতে একটি Java প্রোগ্রাম লিখুন।

✎ **যে পরীক্ষায় এসেছে:** 46th BCS CSE

```
class Calc {                                  // compile time
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}
// Animal, Dog, Cat with sound() overridden: sheet 13
Calc c = new Calc();
System.out.println(c.add(2, 3) + " " + c.add(2.5, 1.5) + " " + c.add(1, 2, 3));
Animal[] a = { new Dog(), new Cat(), new Animal() };
for (Animal x : a) x.sound();                 // run time
```

→ 5 4.0 6 · Dog barks · Cat meows · Animal makes a sound

MCQ Which one is a compile time polymorphism? (ক) Virtual function (খ) Operator overloading (গ) Function overriding (ঘ) Dynamic binding → **(খ)** ✎ 49th BCS CSE (2025)

১৭ Method overloading বনাম overriding

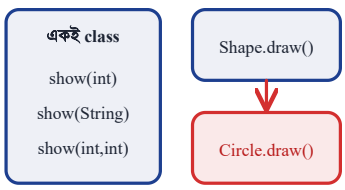
প্র৩৯: উদাহরণসহ method overloading ও method overriding ব্যাখ্যা / সংজ্ঞায়িত করুন।
✎ **যে পরীক্ষায় এসেছে:** 17th NTRCA ICT (2023) · 36th BCS (2016) · BPSC AME 2021 · BPSC AP 2023 · Combined Bank AHE 2021 · Combined Bank SO(AME) 2021 · NU AP 2020 · RAKUB Programmer 2021

প্র৪০: Method Overloading এবং Method Overriding-এর মধ্যকার পার্থক্য লিখুন। / Overloading এবং overriding-এর মধ্যে পার্থক্য কী? / উদাহরণসহ polymorphism এবং overloading ও overriding পদ্ধতি ব্যাখ্যা করুন। / উদাহরণসহ function overloading ও function overriding-এর পার্থক্য দেখান।
✎ **যে পরীক্ষায় এসেছে:** Parliament Secretariat Programmer 2023 · MoFood Network/Website Manager 2025 · 15th NTRCA · 15th NTRCA ICT · 46th BCS ICT

বিষয়	Overloading	Overriding
কী	একই নাম, **ভিন্ন parameter**	একই নাম, **একই signature**, নতুন body
কোথায়	সাধারণত **একই class-এ**	**subclass-এ** (inheritance লাগে)
নির্ধারণ	**compile time** (static)	**run time** (dynamic)
return type	ভিন্ন হতে পারে (তবে একা নয়)	একই (Java-তে subtype-ও)
access	যেকোনো	দুর্বল করা যায় না
static / private / final	overload করা যায়	override করা যায় না
C++ keyword	নেই	base-এ `virtual`
উদ্দেশ্য	সুবিধা: এক ধারণার এক নাম	inherit করা আচরণ নির্দিষ্ট করা

```
class Printer {                               // overloading
    void show(int n) { System.out.println("int " + n); }
    void show(String s) { System.out.println("String " + s); }
    void show(int a, int b) { System.out.println("two " + (a + b)); }
}
class Shape { void draw() { System.out.println("Shape"); } }
class Circle extends Shape {                  // overriding
    @Override void draw() { System.out.println("Circle"); }
}
// main():
Printer p = new Printer();
p.show(5);  p.show("hi");  p.show(2, 3);      // compile time
```

→ int 5
→ String hi
→ two 5
→ Circle



পাশাপাশি বনাম উপর থেকে নিচে

```
Shape s = new Circle();
s.draw();                                   // run time
```

মনে রাখার কৌশল: Over**load** = এক নামে বেশি **বোঝা** (parameter list), পাশাপাশি। Over**ride** = child parent-এর method-এর **উপর চড়ে বসে**, উপর থেকে নিচে।

পরীক্ষার ফাঁদ: শুধু **return type** বদলানো কোনোটিই নয়: এটি compile error ("method already defined")।

১৮ Function ও operator overloading

প্র৪১: Function Overloading কী? উদাহরণসহ ব্যাখ্যা করুন। / ফাংশন ওভারলোডিং ও অপারেটর ওভারলোডিং বলতে কী বুঝেন? উদাহরণ দিন। / Compile time polymorphism কী? কীভাবে অর্জিত হয় (overloading)? কোডসহ উদাহরণ দিন।
✎ **যে পরীক্ষায় এসেছে:** 18th NTRCA College ICT (2024) · 40th BCS (2020) · 16th NTRCA CSE (2019) · BUET M.Sc.

Function overloading: একাধিক function-এর নাম এক, কিন্তু parameter-এর **সংখ্যা, type বা ক্রম** আলাদা; call দেখে compiler একটি বেছে নেয়। **Operator overloading**: `operator+` লিখে কোনো operator-কে (+, ==, <<) class-এর জন্য অর্থ দেওয়া। দুটিই **compile time polymorphism**।

```
int area(int s) { return s * s; }                // square
int area(int l, int w) { return l * w; }         // rectangle
double area(double r) { return 3.1416 * r * r; }  // circle
class Complex {
    double re, im;
public:
    Complex(double r = 0, double i = 0) { re = r; im = i; }
    Complex operator+(const Complex &c) { return Complex(re + c.re, im + c.im); }
    friend ostream& operator<<(ostream &os, const Complex &c)
        { return os << c.re << "+" << c.im << "i"; }
};
cout << area(4) << " " << area(3, 5) << " " << area(1.0);
Complex a(1, 2), b(3, 4);   cout << a + b;
```

→ 16 15 3.1416
→ 4+6i

MCQ A function having more than one distinct meaning is called ___ function. ক) Parameter খ) Prototype গ) Overloaded ঘ) Polymorphism → **গ) Overloaded** ✎ BPSC AP 2022

প্র৪২: `int f(int a, int b = 5);` ও `int f(int &a);` overloaded function দুটি কেন স্বভাবতই দ্ব্যর্থক?
✎ **যে পরীক্ষায় এসেছে:** 38th BCS (2018)

`int x; f(x);`-এ দুটিই সমান মেলে: প্রথমটি **default argument** দিয়ে (`b` = ৫), দ্বিতীয়টি `x`-এ reference বেঁধে। Call-এ দুটি একই দেখায়, তাই compiler থামে: error: call to 'f' is ambiguous.

পরীক্ষার ফাঁদ: ছাপা সমাধান বলে `int b = 5` "প্রশ্নে থাকা উচিত নয়"। ভুল: ঠিক এই default argument-এর **কারণেই** এক-argument-এর call প্রথম function-এ পৌঁছায়।

যেভাবে MCQ এসেছে	উত্তর	যে পরীক্ষায়
Member নয়, global function হিসেবে overload? postfix ++ / comparison / "arithmetic" << / prefix ++	<< (insertion)	BB AD(ICT) 2025
X has `operator*(X)`, `operator*(int)`, `operator+(X)`. কোনটি কম্পাইল হবে? (a) ob2 + 2*ob3 (b) ob2*2 + ob3 (c) (ob2 + 2)*ob3	**(b)**	BUET M.Sc. April 2021

`cout << obj`: বাঁয়ে `ostream`, তাই `<<` আমাদের class-এর member হতে পারে না। (a)-তে লাগে `int*X`, (c)-তে `X+int`: কোনোটিই নেই (compiler দিয়ে যাচাই)।

১৯ Static বনাম dynamic binding, virtual function

প্র৪৩: Object Oriented Programming-এ Static binding ও Dynamic binding কী? ব্যাখ্যা করুন। / উদাহরণসহ early binding ও late binding ব্যাখ্যা করুন।

✎ **যে পরীক্ষায় এসেছে:** BPSC AP 2021 · 36th BCS (2016)

	Static (early) binding	Dynamic (late) binding
call ও body জোড়া লাগে	**compile time**-এ	**run time**-এ
কে ঠিক করে	**pointer / reference**-এর type	যে **object**-কে নির্দেশ করে তার type
যেখানে	সাধারণ, overloaded, static, private function	`virtual` function (C++), override করা method (Java)

```
class Shape { public:
    virtual void draw() { cout << "Shape\n"; }
    void name() { cout << "shape\n"; }
    virtual ~Shape() {} };
class Circle : public Shape { public:
    void draw() override { cout << "Circle\n"; }
    void name() { cout << "circle\n"; } };
Shape* p = new Circle;
p->draw();     // virtual: late binding
p->name();     // not virtual: early binding
```

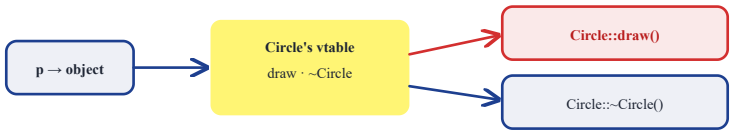
→ Circle
→ shape

একই pointer, দুই উত্তর: শুধু `virtual` call আসল object দেখে।

প্র৪৪: Virtual function কী? উদাহরণসহ ব্যাখ্যা করুন।

✎ **যে পরীক্ষায় এসেছে:** BITAC AP 2023

Virtual function হলো base class-এর `virtual` দিয়ে ঘোষিত member function, যা derived class-এ নতুন করে লেখা হয়; base pointer বা reference দিয়ে ডাকলে **আসল object**-এর সংস্করণ চলে (উপরের উদাহরণ)। নিয়ম: non-static member, derived class-এ একই signature, base-এ virtual destructor। **Pure virtual function**-এর body নেই (`= 0`), ফলে class হয় **abstract**: `Shape s;` লিখলে error "variable type 'Shape' is an abstract class"।



late binding: object → তার class-এর vtable → সঠিক body

যেভাবে MCQ এসেছে	উত্তর	যে পরীক্ষায়
C++-এ কোনটি pure virtual function ঘোষণা করে? `virtual void f() = 0;` / `virtual void f();` / `void f() = 0;` / `pure virtual f();`	`virtual void f() = 0;`	49th BCS ICT (2025)
Java-তে কোন keyword method override হওয়া ঠেকায়? const / final / static / synchronized	**final**	49th BCS ICT (2025)

২০ Interface, এবং override / hide ধাঁধা

প্র৪৫: Object Oriented Programming-এ Interface কী কাজে লাগে?

✎ **যে পরীক্ষায় এসেছে:** 16th NTRCA ICT

Interface হলো বিশুদ্ধ চুক্তি: state ছাড়া শুধু method signature। কাজ: ① **abstraction** — ব্যবহারকারী নির্ভর করে *কী*-র উপর, *কীভাবে*-র উপর নয় · ② একটি class **অনেক interface** implement করতে পারে, Java-র নিরাপদ multiple inheritance · ③ **loose coupling**: implementation বদলানো যায় (`List` → `ArrayList` / `LinkedList`) · ④ interface reference দিয়ে run-time polymorphism।

প্র৪৬: Interface-এর abstract method implement করুন: এমন child class লিখুন যাতে এই `main` চলে। `interface Fly { void print(); } ... Fly dog = new Dog(); Fly bird = new Bird(); dog.print(); bird.print();`

✎ যে পরীক্ষায় এসেছে: BCIC AP 2025

```
class Dog implements Fly {
    public void print() { System.out.println("Dog cannot fly"); } }
class Bird implements Fly {
    public void print() { System.out.println("Bird can fly"); } }
```

→ Dog cannot fly
→ Bird can fly

প্রশ্নপত্রের প্রত্যাশিত আউটপুট সংরক্ষিত নেই, তাই যেকোনো দুটি লাইন প্রিন্ট করুন। Interface-এর method নিজে থেকেই `public`: `public` বাদ দিলে compile error।

প্র৪৭: Overloaded, overridden ও hidden superclass method নির্ণয় করুন: `class A { public void m1() {} public void m2() {} public void m3(int i) {} public void m4(String i) {} } class B extends A { public static void m1() {} public void m2() {} public void m3(int i) {} public static void m4(String i) {} }`

✎ যে পরীক্ষায় এসেছে: BB AP 2023

Overridden: `m2()`, `m3(int)` (একই signature, instance method)। **Overloaded:** নেই (একই নামে ভিন্ন parameter নেই)। **Hidden:** ছাপা অবস্থায় **নেই — কম্পাইলই হয় না**: javac বলে "m1() in B cannot override m1() in A", কারণ static method কোনো **instance** method লুকাতে পারে না। Parent-এর method-ও static হলে তবেই hiding হয়:

```
class A { public static void m1() { System.out.println("A.m1"); } ... }
class B extends A { public static void m1() { System.out.println("B.m1"); } ... }
A ref = new B();
ref.m1(); ref.m2(); ref.m3(1); ref.m4("x");    // m1, m4 static in both
```

→ A.m1 · B.m2 · B.m3 · A.m4

Static = **hidden**, reference-এর type দেখে বাছাই; instance = **overridden**, object দেখে বাছাই।

২১ কোডের প্রশ্ন: ++, <<, default, lambda

প্র৪৮: `Integer { private: int i; }` class-এর জন্য C++-এ operator overloading function লিখুন যাতে `main() { Integer P; P++; ++P; }` চলে।

✎ যে পরীক্ষায় এসেছে: BUET AP 2025

```
class Integer {
    int i;
public:
    Integer(int v = 0) { i = v; }
    Integer& operator++() { ++i; return *this; }      // prefix ++P
    Integer operator++(int) {                          // postfix P++
        Integer old = *this; ++i; return old; }        // dummy int
    int get() const { return i; }
```

→ a = 0, b = 2, P = 2

dummy int parameter শুধু compiler-কে জানায় "এটি postfix"; postfix কপি করে **পুরোনো** মান ফেরত দেয়।

```
};
Integer P;  Integer a = P++;  Integer b = ++P;
```

প্র৪৯: Java-তে functional interface কী? উদাহরণ দিন।

✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. April 2024

```
@FunctionalInterface
interface Square { int calc(int x); }     // exactly one abstract method
Square s = x -> x * x;                    // a lambda supplies it
System.out.println(s.calc(7));
```

→ 49

যে interface-এ **ঠিক একটি abstract method** থাকে (default/static method চলে), তাই lambda দিয়ে implement করা যায়: `Runnable`, `Comparator`।

যেভাবে MCQ এসেছে	উত্তর	যে পরীক্ষায়
`interface Alpha { default void reset(){} }, interface Beta { default void reset(){} }, class Gamma implements Beta, Alpha {}`: কোনটি error ঠিক করে?	**interface Beta extends Alpha**	BUET M.Sc. April 2021
In `class G<A extends X & Y & Z>`: (a) X, Y class, Z interface (b) X, Y interface, Z class (c) সবই interface (d) সবই class	**(c)**	BUET M.Sc. April 2021

দুটি সম্পর্কহীন default-এ সংঘাত হয় ("types Beta and Alpha are incompatible"); `Beta extends Alpha` হলে Beta-র সংস্করণ জেতে। Bound-এ শুধু **প্রথম** type class হতে পারে, বাকিগুলো interface হতেই হবে, তাই অপশনগুলোর মধ্যে শুধু (c) কম্পাইল হয়।

পরীক্ষার ফাঁদ: `virtual` ছাড়া `void f() = 0;` compile error; আর `static` method কখনো virtual হয় না।

২২ Constructor ও destructor

প্র৫০: Constructor function কী? এর বৈশিষ্ট্যগুলো লিখুন। / Constructor function-এর একটি উদাহরণ লিখুন। / OOP-তে constructor কখন invoke হয়?

✎ **যে পরীক্ষায় এসেছে:** WZPGCL AE(CSE) 2023 · WZPDCL AE(CSE) 2019 · BPSC AP 2022

Constructor: **class-এর নামেই** একটি বিশেষ member function, যা **object তৈরির সময় নিজে থেকে চলে** এবং তাকে initialise করে। প্রতিটি object তৈরিতে এটি **invoke** হয়: ঘোষণা `Point p;`, `new Point()`, কপি, বা value দিয়ে object পাঠালে।

- ✓ class-এর নামেই নাম
- ✓ **return type নেই**, `void`-ও নয়
- ✓ নিজে থেকে চলে, প্রতি object-এ একবার
- ✓ **overload** করা যায়; default argument নিতে পারে
- ✓ inherit হয় না; `virtual` (C++) বা `static`/`final`/`abstract` (Java) হতে পারে না
- ✓ একটিও না লিখলে compiler একটি default দেয়

→ 0,0 3,4 (প্রকার: default · parameterised · copy)

```
class Point {
    int x, y;
public:
    Point() { x = y = 0; }                  // default
    Point(int a, int b) { x = a; y = b; } // parameterised
    void show() { cout << x << "," << y << " "; }
};
Point p;  Point q(3, 4);  p.show();  q.show();
```

প্র৫১: OOP-তে constructor ও destructor function কেন ব্যবহৃত হয়? C++ বা Java-তে প্রতিটির উদাহরণ দিন। / উদাহরণসহ destructor ও constructor-এর মূল পার্থক্য কী? / Class ও Object কী? Constructors ও Destructors ফাংশনের মধ্যে পার্থক্য লিখুন। / Constructor, destructor ও copy constructor-এর সংজ্ঞা দিন।

✎ **যে পরীক্ষায় এসেছে:** BPSC SAE 2021 · PSB Programmer 2018 · 17th NTRCA CSE (2023) · 47th BCS ICT

	Constructor	Destructor
কাজ	object **initialise** করা, resource নেওয়া	**পরিষ্কার** করা, মেমরি / ফাইল ছেড়ে দেওয়া
নাম	`Point()`	`~Point()` (tilde)
কখন চলে	object তৈরির সময়	scope থেকে বেরোলে / `delete` করলে
argument	নিতে পারে; overload করা যায়	**নেই**; প্রতি class-এ ঠিক একটি
inheritance-এ ক্রম	base → derived	derived → base

২৩ কাজে destructor, copy constructor

এক রানে constructor + destructor (আগের পাতার প্রশ্নগুলোর জন্য):

```
class Logger {
    string name;
public:
    Logger(string n) : name(n) { cout << "open " << name << "\n"; }
    ~Logger() { cout << "close " << name << "\n"; }
};
int main() {
    Logger a("A");
    { Logger b("B"); }          // b dies at this brace
    cout << "end of main\n";
}
```

→ open A · open B · close B · end of main · close A

Copy constructor `Point(const Point &p)` বিদ্যমান object থেকে নতুন object বানায় (নিচে)। Java-তে **destructor নেই**: garbage collector মেমরি ছাড়ে; ফাইলের জন্য `try-with-resources`।

প্র৫২: C++-এ copy constructor ব্যবহার করে অন্য object থেকে member variable initialise করার একটি প্রোগ্রাম লিখুন।
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. May 2016

```
class Student {
    string name; int roll;
public:
    Student(string n, int r) { name = n; roll = r; }
    Student(const Student &s) {              // copy constructor
        name = s.name; roll = s.roll;
        cout << "copy constructor called\n"; }
    void show() { cout << name << " " << roll << "\n"; }
};
void print(Student s) { s.show(); }          // by value
int main() {
    Student s1("Rafi", 7);
    Student s2 = s1;                         // from another object
    s2.show();   print(s1);
}
```

→ copy constructor called
→ Rafi 7
→ copy constructor called
→ Rafi 7

Parameter হলো **reference**: `Student(Student s)` হলে কপি বানাতে আবার কপি লাগত — অসীম recursion, তাই compiler এটি বাতিল করে।

MCQ If object is passed by value: (a) copy constructor মান কপি করে একটি temporary object-এ (b) copy constructor মান কপি করে function-এর ভেতরের আরেকটি object-এ (c) object-এর reference ব্যবহৃত হয় (d) reference তার জায়গায় নতুন object বানায় → **(b)**: উপরের দ্বিতীয় "copy constructor called" হলো `print`-এর ভেতরের parameter `s`। ✎ BUET IICT April 2021

MCQ `class A { int a; public: A(int a) { this->a = a; } }; class B : A { int b; public: ___ };` B-এর জন্য বৈধ constructor কোনটি? (a) `B(int a, int b) : A(a), b(b) { }` (b) `B(int a, int b) { super(a); this->b = b; }` (c) `B(int a, int b) : A(a), this->b(b) { }` (d) none → **(a)**; (b) Java-র syntax এবং A-র default constructor নেই, (c) বৈধ initialiser নয়। ✎ BUET M.Sc. April 2021

২৪ Default constructor, `static`, list-এ object

প্র৫৩: C++ কোড দেওয়া আছে: কোডটি কম্পাইল হতে class-এ কী লাগবে তা লিখুন (constructor)।
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. October 2018

সাধারণ ঘটনা: class-এ শুধু `Point(int a, int b)`, আর `main`-এ লেখা `Point p;`। Error "no matching constructor for initialization of 'Point'": যেকোনো constructor লিখলেই বিনা মূল্যের default-টি আর থাকে না। যোগ করুন **`Point() {}`** বা default দিন: `Point(int a = 0, int b = 0)`।

প্র৫৪: Java-তে একটি object কতবার instantiate হয়েছে তা বের করার method লিখুন। / 'Static' কীওয়ার্ডটি ব্যাখ্যা করার জন্যে Static variable এবং Static method ব্যবহার করে একটি প্রোগ্রাম লিখুন। / Class-এর static method ডাকার syntax কী?
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. May 2016 · 15th NTRCA ICT · BCC AP 2023

```
class Item {
    private static int count = 0;      // one copy for all objects
    Item() { count++; }                // every constructor call counts
    static int getCount() { return count; }
}
new Item(); new Item(); new Item();
System.out.println(Item.getCount());   // ClassName.method()
```

→ 3

Static member **class-এর**, কোনো object-এর নয়: একটিই শেয়ার করা কপি, object ছাড়াই ব্যবহার করা যায়। ডাকুন `ClassName.method()` (Java) বা `ClassName::method()` (C++) দিয়ে। Static method `this` ব্যবহার করতে পারে না।

প্র৫৫: Constructor, setter ও getter-সহ একটি Java class লিখুন; এর ৭টি object তৈরি করে ArrayList-এ রাখুন।
✎ **যে পরীক্ষায় এসেছে:** BPDB AE(CSE) 2018

```
class Book {
    private String title;  private int price;
    Book(String title, int price) { this.title = title; this.price = price; }
    String getTitle() { return title; }   int getPrice() { return price; }
    void setPrice(int price) { this.price = price; }
}
ArrayList<Book> list = new ArrayList<>();
for (int i = 1; i <= 7; i++) list.add(new Book("Book" + i, 100 * i));
list.get(0).setPrice(150);
```

→ size 7 · প্রথমটির price 150 · শেষটি Book7

যেভাবে MCQ এসেছে	উত্তর	যে পরীক্ষায়
Which of the following do not return any value? ক) Constructor function খ) Friend function গ) Inline Function ঘ) Member Function	**constructor**	BPSC AP 2022
A constructor is a special type of- A. Class B. Field C. Method D. Property	**method**	BPSC AME 2019
Java-তে object তৈরি করতে কী invoke হয়?	**constructor**	BPDB AE(CSE) 2021

২৫ C++ বনাম Java, processor, data type

প্র৫৬: C++ ও Java প্রোগ্রামিং ভাষার পার্থক্য কী কী?
✎ **যে পরীক্ষায় এসেছে:** 36th BCS (2016)

বিষয়	C++	Java
যেভাবে চলে	native machine code	**JVM-এ bytecode**
platform	প্রতি platform-এ আবার compile	একবার লিখে যেকোনো জায়গায় চালান
pointer	আছে	নেই (শুধু reference)
মেমরি	`new`/`delete`, destructor	**garbage collector**
multiple inheritance	class-এর	শুধু interface দিয়ে
operator overloading	আছে	নেই
generic কোড	template	generics
global function	চলে	সবকিছু class-এর ভেতরে

প্র৫৭: Python, C, C++ ও Java-র language processor-এর ধরন লিখুন।
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. May 2016

C, C++: **compiler** → native machine code। **Java**: compiler (`javac` → bytecode) **এবং** JVM, যা interpret ও JIT-compile করে: একটি **hybrid**। **Python**: **interpreter** (CPython আগে bytecode বানায়, তারপর interpret করে)।

প্র৫৮: C++ প্রোগ্রামিং ভাষায় বিভিন্ন ডাটা টাইপ বর্ণনা করুন।
✎ **যে পরীক্ষায় এসেছে:** 13th NTRCA CSE (2016)

শ্রেণি	type (৬৪-বিট compiler-এ আকার, `sizeof` দিয়ে ছাপা)
built-in	`char` 1 · `bool` 1 · `short` 2 · `int` 4 · `long long` 8 · `float` 4 · `double` 8 · `void`
derived	array, pointer, reference, function
user-defined	`class`, `struct`, `union`, `enum`, `typedef`/`using`

প্র৫৯: একটি Java Swing কোড lambda expression দিয়ে আবার লিখুন।
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. October 2017

```
b.addActionListener(new ActionListener() {                    // before: anonymous class
    public void actionPerformed(ActionEvent e) { System.out.println("clicked"); }
});
b.addActionListener(e -> System.out.println("clicked"));      // after: lambda
```

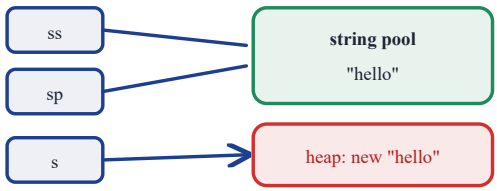
`ActionListener`-এ একটিই abstract method, তাই এটি functional interface এবং lambda anonymous class-এর জায়গা নিতে পারে (দুটিই কম্পাইল হয়; বোতামে তখন ২টি listener)।

মনে রাখার কৌশল: Java = "**C++ বিয়োগ**": pointer বাদ, `delete` বাদ, operator overloading বাদ, class-এর multiple inheritance বাদ।

২৬ আউটপুট বের করুন: string, char, assignment

প্র৬০: আউটপুট বের করুন: `String s = new String("hello"); String ss = "hello"; String sp = "hello"; print(s.compareTo(ss)); print(s == ss); print(s == sp); print(ss == sp);` / String-সহ Java কোড দেওয়া আছে, আউটপুট কী?
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. May 2019

→ 0 (একই অক্ষর)
→ false
→ false
→ true
(আর `s.equals(ss)` → true)



`==` তুলনা করে **reference**; `equals()` / `compareTo()` তুলনা করে **বিষয়বস্তু**। Literal-গুলো **string pool**-এ একটি object শেয়ার করে; `new String` নতুন object বানায়। (BUET ২০১৯-এর দ্বিতীয় string প্রশ্নটি সংরক্ষিত নেই, একই নিয়ম যাচাই করে।)

যেভাবে MCQ এসেছে	উত্তর	যে পরীক্ষায়
`System.out.print('D' + 'E' + 'F');` (a) 137 (b) DEF (c) 207 (d) D E F	**(c) 207**	Combined Bank AHE/AE(IT) 2021
`int a=10, b=15; if((a=100)==(b=a)) print(b+a); else print(b-a);` (a) 100 (b) 200 (c) 0 (d) 3	**(b) 200**	Combined Bank AHE/AE(IT) 2021
String-এর বৈধ ArrayList: (a) `List<String> list = new ArrayList<>();` (b) `List<>` … (c), (d) `new List<>()`	**(a)**	BUET M.Sc. April 2021

`char + char` হলো **int** যোগ: 68 + 69 + 70 = **207**; `"" + 'D' + 'E' + 'F'` হলে DEF ছাপত। দ্বিতীয়টিতে `a` হয় 100, তারপর `b = a` = 100, তাই শর্ত সত্য এবং `b + a` = 200।

পরীক্ষার ফাঁদ: ছাপা উত্তর (a) 100; প্রোগ্রাম চালালে ছাপে **200**।

প্র৬১: Java কোড দেওয়া আছে: for loop ছাড়া string list-এর প্রতিটি string-এর দৈর্ঘ্য বের করুন।
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. October 2018

```
List<String> list = List.of("java", "c", "python");
list.stream().map(String::length).forEach(System.out::println);
list.forEach(s -> System.out.println(s + " " + s.length()));
```

→ 4 · 1 · 6
→ java 4 · c 1 · python 6

প্র৬২: একটি Java প্রোগ্রামের আউটপুট বের করুন (প্রোগ্রামটি কোনো কপিতে সংরক্ষিত নেই)।
✎ **যে পরীক্ষায় এসেছে:** WZPDCL AE(CSE) 2019

এই পাতার চারটি আউটপুট দিয়ে অনুশীলন করুন।

২৭ OOP প্রশ্নপত্রে আসা ছোট প্রোগ্রাম

প্র৬৩: যেকোনো প্রোগ্রামিং ভাষায় এমন একটি প্রোগ্রাম লিখুন যা keyboard থেকে পাঁচটি সংখ্যা পড়ে সবচেয়ে ছোট, সবচেয়ে বড় ও গড় দেখায়।
✎ **যে পরীক্ষায় এসেছে:** BDB SO(IT) 2017

```
// inside main(); import java.util.Scanner
Scanner sc = new Scanner(System.in);
double min = Double.MAX_VALUE, max = -Double.MAX_VALUE, sum = 0;
for (int i = 0; i < 5; i++) {
    double x = sc.nextDouble();
    sum += x;  if (x < min) min = x;  if (x > max) max = x;
}
System.out.println("Smallest " + min + ", largest " + max
                   + ", average " + sum / 5);
```

input 4 9 -2 7.5 1 → Smallest -2.0, largest 9.0, average 3.9

পরীক্ষার ফাঁদ: `max` শুরু করুন `-Double.MAX_VALUE` দিয়ে, `Double.MIN_VALUE` দিয়ে নয়: MIN_VALUE হলো ক্ষুদ্রতম **ধনাত্মক** double, তাই সব ইনপুট ঋণাত্মক হলে ভুল সর্বোচ্চ ছাপবে।

প্র৬৪: C++-এ একটি প্রোগ্রাম লিখুন যা প্রদত্ত সংখ্যা মৌলিক কি না যাচাই করে।

✎ **যে পরীক্ষায় এসেছে:** 47th BCS ICT

```
#include <iostream>
using namespace std;
bool isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++)        // up to √n is enough
        if (n % i == 0) return false;
    return true;
}
int main() {
    int n;  cin >> n;
    cout << n << (isPrime(n) ? " is prime" : " is not prime");
}
```

97 → 97 is prime
91 → 91 is not prime (7 × 13)

n-এর √**n**-এর চেয়ে বড় কোনো উৎপাদক থাকলে ছোট একটিও থাকে, তাই loop থামে `i*i <= n`-এ।

প্র৬৫: এমন একটি Java প্রোগ্রাম লিখুন যা তিনটি ইনপুট নিয়ে প্রত্যেকটির উৎপাদক বের করে।

✎ **যে পরীক্ষায় এসেছে:** NHA AP 2019

```
static void factors(int n) {
    System.out.print("Factors of " + n + ":");
    for (int i = 1; i <= n; i++) if (n % i == 0) System.out.print(" " + i);
    System.out.println();
}
// main(): Scanner sc = new Scanner(System.in);
//         for (int k = 0; k < 3; k++) factors(sc.nextInt());
```

input 12 7 20
→ Factors of 12: 1 2 3 4 6 12
→ Factors of 7: 1 7
→ Factors of 20: 1 2 4 5 10 20

প্র৬৬: "কোড অংশটির আউটপুট কী?" / "C++ প্রোগ্রাম" (কোড বা কাজটি কোনো কপিতে সংরক্ষিত নেই)।

✎ **যে পরীক্ষায় এসেছে:** BCIC AP 2022 · DUET AP 2020

পাতা ১৩, ২০, ২৪ ও ২৭-এর ট্রেস করা প্রোগ্রামগুলো অনুশীলন করুন: constructor-এর ক্রম, virtual call, copy constructor, ==।

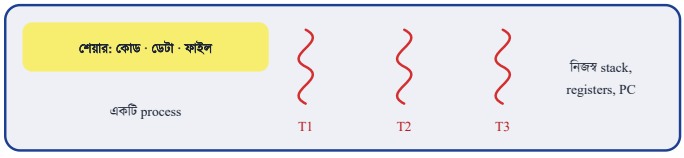
২৮ Thread, multithreading, multitasking

প্র৬৭: থ্রেড (Thread)-এর সংজ্ঞা দিন। / Multithreading programming কী? প্রোগ্রামিংয়ে multithreading কেন ব্যবহৃত হয়?

✎ **যে পরীক্ষায় এসেছে:** 15th NTRCA · Combined Bank AP 2024

Thread: OS যে ক্ষুদ্রতম execution একক schedule করে: নির্দেশনার একটি ধারা, যার **নিজস্ব stack, register ও program counter** আছে, আর নিজের **process**-এর কোড, ডেটা ও খোলা ফাইল অন্য thread-এর সঙ্গে শেয়ার করে।

Multithreading = একটি প্রোগ্রাম একসঙ্গে কয়েকটি thread চালায় (একাধিক core-এ সত্যিকারের সমান্তরাল, এক core-এ পালাক্রমে)। **কেন:** ① download চলার সময়ও UI সচল · ② multi-core CPU-তে গতি · ③ thread মেমরি শেয়ার করে, process-এর চেয়ে সস্তা · ④ server অনেক client সামলায়। **খরচ:** শেয়ার করা ডেটায় **synchronisation** লাগে, নইলে race condition হয় (overdraft account, পরের পাতা)।



একটি process-এর ভেতরে তিনটি thread

প্র৬৮: Multitasking ও Multiprocessing-এর পার্থক্য লিখুন।
✎ **যে পরীক্ষায় এসেছে:** DESCO SAE 2023

	Multitasking	Multiprocessing
অর্থ	কয়েকটি কাজ time-slicing-এ **একটি CPU** ভাগ করে	কয়েকটি **CPU / core** সত্যিকারের সমান্তরালে কাজ চালায়
সমান্তরালতা	আপাত (দ্রুত বদল)	প্রকৃত
hardware	একটি processor-ই যথেষ্ট	২ বা ততোধিক processor লাগে
লক্ষ্য	সাড়া দেওয়া, অনেক user/app	throughput, গতি

প্র৬৯: C++-এ thread দিয়ে factorial হিসাব করুন (pthread নয়)।
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. October 2017

```
#include <iostream>
#include <thread>
using namespace std;
void factorial(int n, unsigned long long &result) {
    result = 1;
    for (int i = 2; i <= n; i++) result *= i;
}
int main() {
    unsigned long long r1, r2;
    thread t1(factorial, 10, ref(r1));     // std::thread
    thread t2(factorial, 20, ref(r2));
    t1.join();  t2.join();                 // wait for both
    cout << "10! = " << r1 << "\n20! = " << r2;
}
```

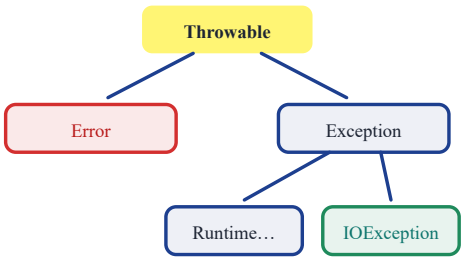
→ 10! = 3628800
→ 20! = 2432902008176640000

Thread-এ reference পাঠাতে `std::ref` লাগে; `join()` অপেক্ষা করে। ৬৪ বিটে ধরে এমন বৃহত্তম factorial 20!।

২৯ Exception, এবং thread-safe account

প্র৭০: Java-তে exception ও error-এর পার্থক্য কী? / Exception handling কী? উদাহরণসহ লিখুন।
✎ **যে পরীক্ষায় এসেছে:** SPCBL SAE 2022

Exception	Error
প্রোগ্রাম **সামলাতে পারে**	গুরুতর JVM / system ত্রুটি
`try-catch` দিয়ে সামলানো যায়	catch করা **উচিত নয়**
checked বা unchecked	সবসময় unchecked
`FileNotFoundException`	`OutOfMemoryError`, `StackOverflowError`



Exception handling run-time সমস্যা ধরে, যাতে প্রোগ্রাম crash না করে: `try` (যে কোড ব্যর্থ হতে পারে) → `catch` (সামলানো) → `finally` (সবসময় চলে: ফাইল বন্ধ) · `throw` exception ছোড়ে · `throws` ঘোষণা করে।

```
int[] a = {10, 20};
try {
    System.out.println(a[0] / 0);
} catch (ArithmeticException e) {
    System.out.println("Caught: " + e.getMessage());
} finally {
    System.out.println("finally always runs");
```

→ Caught: / by zero
→ finally always runs
→ program continues

```
}
System.out.println("program continues");
```

প্র৭১: Java-তে "Overdraft Account (OD)" class implement করুন: অনুমোদিত ঋণসীমা (যেমন ১০০০০০) দিয়ে খোলা হয়; যেকোনো সময় যেকোনো অঙ্ক জমা; সীমার মধ্যে থাকলে balance ছাড়িয়েও তোলা যায়; কোড multithreading mode-এ চলবে। (উৎসে বাকি অংশ কাটা।)

✎ **যে পরীক্ষায় এসেছে:** BB AP 2019

```
class ODLimitExceeded extends Exception { ODLimitExceeded(String msg) { super(msg); } }
class ODAccount {
    private double balance = 0;  private final double limit;
    ODAccount(double limit) { this.limit = limit; }
    synchronized void deposit(double amt) { balance += amt; }
    synchronized void draw(double amt) throws ODLimitExceeded {
        if (balance - amt < -limit) throw new ODLimitExceeded("limit " + limit + " exceeded");
        balance -= amt; }
    synchronized double getBalance() { return balance; }
}
ODAccount acc = new ODAccount(100000);
Runnable job = () -> { for (int i = 0; i < 1000; i++) acc.deposit(10); };
Thread t1 = new Thread(job), t2 = new Thread(job);  t1.start(); t2.start(); t1.join(); t2.join();
try { acc.draw(120000); System.out.println(acc.getBalance()); acc.draw(1); }
catch (ODLimitExceeded e) { System.out.println("Refused: " + e.getMessage()); }
```

→ 20000.0 (সবসময়: `synchronized`) · -100000.0 · Refused: limit 100000.0 exceeded

৩০ Facade pattern, এবং কোম্পানির মডেল

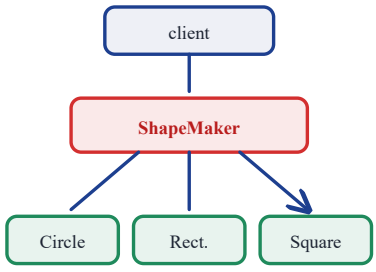
প্র৭২: A `Shape` interface with `Circle`, `Rectangle`, `Square`; একটি `ShapeMaker` class ও `FacadePatternDemo`: pattern-এর নাম লিখুন ও implement করুন।

✎ **যে পরীক্ষায় এসেছে:** BPDB AE(CSE) 2023

এটি **Facade pattern** (structural): একগুচ্ছ class-এর সামনে একটি সহজ class (`ShapeMaker`), তাই client `drawCircle()` ডাকে, কোন object কাজটি করে তা না জেনেই।

```
interface Shape { void draw(); }
class Circle implements Shape {
    public void draw() { System.out.println("Circle::draw()"); } }
class Rectangle implements Shape {
    public void draw() { System.out.println("Rectangle::draw()"); } }
class Square implements Shape {
    public void draw() { System.out.println("Square::draw()"); } }
class ShapeMaker {                                  // the facade
    private Shape circle = new Circle(), rectangle = new Rectangle(),
                  square = new Square();
    public void drawCircle()    { circle.draw(); }
    public void drawRectangle() { rectangle.draw(); }
    public void drawSquare()    { square.draw(); }
}
public class FacadePatternDemo {
    public static void main(String[] args) {
        ShapeMaker m = new ShapeMaker();
        m.drawCircle(); m.drawRectangle(); m.drawSquare();
```

→ Circle::draw()
→ Rectangle::draw()
→ Square::draw()



```
    }
}
```

প্র৭৩: একটি কোম্পানির কর্মী, প্রকল্প ও কর্মীর তথ্য রাখার জন্য class লিখুন।
✎ **যে পরীক্ষায় এসেছে:** BUET M.Sc. October 2018

```
class Employee { int id; String name;
    Employee(int id, String name) { this.id = id; this.name = name; } }
class Project { String code; List<Employee> members = new ArrayList<>();
    Project(String code) { this.code = code; }
    void assign(Employee e) { members.add(e); } }
class Company { List<Employee> staff = new ArrayList<>();
    List<Project> projects = new ArrayList<>();
    void hire(Employee e) { staff.add(e); }
    void info(Employee e) {                    // the projects of one employee
        StringBuilder sb = new StringBuilder(e.name + ":");
        for (Project p : projects) if (p.members.contains(e)) sb.append(" ").append(p.code);
        System.out.println(sb); } }
```

Sami: ERP + Web, Lina: Web → Sami: ERP Web → Lina: Web

Company-র **আছে** কর্মী ও প্রকল্প (**composition**, HAS-A, IS-A নয়); কর্মী ↔ প্রকল্প many-to-many।

★ পরীক্ষার আগের রাত

১ **স্তম্ভ: A PIE** — abstraction, polymorphism, inheritance, encapsulation; "তিন মূলনীতি" = E, I, P।

২ **Overloading** = একই নাম, ভিন্ন parameter, compile time · **overriding** = subclass-এ একই signature, run time (৮টি পরীক্ষা)।

৩ **Friend function**: private ডেটায় access-সহ non-member; `this` নেই; encapsulation ভাঙে।

৪ **Inheritance**: single, multilevel, hierarchical, multiple, hybrid; diamond → `virtual` base। Constructor A → B → C, destructor উল্টো।

৫ **Constructor**: class-এর নাম, return type নেই, নিজে থেকে চলে, overload করা যায়। Destructor: ~, argument নেই, প্রতি class-এ একটি। Copy constructor নেয় `const T&`।

৬ **Access**: public YYYY · protected YYYN · private YNNN; C++ class-এ default private, struct-এ public।

৭ **Virtual**: object দেখে late binding; pure virtual `= 0` → abstract class; Java-তে `final` overriding ঠেকায়।

৮ **Java-র ফাঁদ**: `'D'+'E'+'F'` = 207 · string-এ == reference তুলনা করে · `(a=100)==(b=a)` → 200।

✓ অনুশীলনী MCQ (সবগুলো পরীক্ষায় এসেছে; উত্তর নিচে)

১. কোনটি pure virtual function ঘোষণা করে? (a) `virtual void f();` (b) `virtual void f() = 0;` (c) `void f() = 0;` (d) `pure virtual f();` ✎ 49th BCS ICT (2025)

২. Compile time polymorphism: (a) virtual function (b) operator overloading (c) overriding (d) dynamic binding ✎ 49th BCS CSE (2025)

৩. C++-এ default inheritance: (a) public (b) private (c) protected (d) error ✎ BPSC AME 2019

৪. C : B : A হলে constructor-এর ক্রম — (a) C B A (b) A C B (c) A B C (d) C A B ✎ BUET IICT April 2021

৫. Java-তে overriding ঠেকায়: (a) const (b) final (c) static (d) synchronized ✎ 49th BCS ICT (2025)

৬. `System.out.print('D'+'E'+'F');` (a) 137 (b) DEF (c) 207 (d) D E F ✎ Combined Bank AHE/AE(IT) 2021

৭. কোনটি encapsulation অর্জন করে না: (a) private (b) class (c) getter/setter (d) global variable ✎ BB AD(ICT) 2025

৮. কোনটি কোনো মান return করে না: (a) constructor (b) friend (c) inline (d) member function ✎ BPSC AP 2022

উত্তর: ১ b · ২ b · ৩ b · ৪ c · ৫ b · ৬ c · ৭ d · ৮ a

মনে রাখার কৌশল: যেকোনো OOP উত্তরের আগে: **সংজ্ঞা** → **টেবিল** → **কোড** → **আউটপুট** → **ফাঁদ**। পাঁচ ঘর, পুরো নম্বর।

পরীক্ষার ফাঁদ: এক প্রোগ্রামে C++ ও Java কখনো মেশাবেন না: C++-এ `this.x`, pointer ছাড়া `new` বা `extends` লিখলে নম্বর কাটে।